

SISTEMI AD AGENTI

Caso di Studio:

Wake Word Detection

“Pixie”

Anno Accademico 2025/2026

Documentazione Progetto

1. Introduzione

Il presente documento descrive il progetto di riconoscimento della wake word "Pixie", sviluppato nell'ambito del corso di Sistemi ad Agenti. L'obiettivo principale del progetto è addestrare un modello di rilevamento della parola di attivazione leggero ed eseguibile su Android, anche su dispositivi di fascia bassa. Un requisito importante è la robustezza del sistema: il modello deve riconoscere la wake word evitando il più possibile i falsi positivi che si possono verificare in ambienti rumorosi o con parlato generico.

Il sistema si articola in tre macro-fasi:

- Raccolta e generazione del dataset audio (registrazioni reali + sintesi TTS)
- Addestramento e adattamento di un modello di wake word detection (openWakeWord)
- Validazione e test in scenari reali con rumore di fondo, parlato sovrapposto e casi limite

2. Il Dataset

La qualità del dataset è il fattore più critico per le prestazioni del rilevatore. Il dataset del progetto è organizzato in due macro-categorie: campioni **Positivi** (pronunce di "Pixie", suddivisi in generati da TTS e reali) e campioni **Negativi**, a loro volta distinti in due sottocategorie: *avversi* (parole foneticamente simili a "Pixie", sia generate che reali) e *generici* (parlato generico senza la wake word). I negativi generici non sono singoli samples da 1 secondo, ma derivano da due fonti combinate: le feature openWakeWord pre-calcolate del dataset ACAV100M (~2.000 ore di audio generico, fornite dalla libreria) e circa 10 ore di parlato italiano estratte dal corpus Mozilla Common Voice 26.0, convertite e pre-elaborate localmente per insegnare al modello a rifiutare il parlato italiano "difficile". Le dieci parole usate per i negativi avversi (Pizza, Pixel, Pyrex, Taxi, Dixit, Dixie, Bixby, Pepsi, Mix) sono le stesse proposte ai partecipanti per le registrazioni reali; nella parte sintetica, invece, openWakeWord ne genera autonomamente le varianti tramite il proprio phonemizer, ampliando l'insieme dei negativi avversi a partire dalle parole indicate. Tutte le registrazioni reali, sia positive che negative, sono sottoposte a validazione automatica tramite appositi programmi prima dell'inserimento nel dataset di addestramento.

2.1 Registrazioni Reali

Le registrazioni reali sono state raccolte da più parlanti con l'obiettivo di addestrare il sistema a riconoscere la wake word "Pixie" in condizioni il più possibile realistiche. Per raggiungere un numero ampio e variegato di partecipanti, è stato inviato un messaggio di richiesta tramite WhatsApp a contatti personali. La preferenza per WhatsApp rispetto a soluzioni basate su moduli web (come Google Forms) è motivata dalla semplicità operativa per i partecipanti: registrare e inviare un messaggio vocale dall'app richiede un solo gesto, senza necessità di account aggiuntivi né di passaggi di caricamento file che avrebbero reso la partecipazione più macchinosa e scoraggiato chi è meno pratico con la tecnologia. Nel messaggio veniva spiegato lo scopo del progetto (addestrare un assistente vocale a riconoscere "Pixie" evitando false attivazioni), specificando che i vocali sarebbero stati utilizzati esclusivamente per il riconoscimento vocale. A ciascun partecipante veniva chiesto di registrare 8 messaggi vocali separati seguendo una scaletta precisa: 4 pronunce della wake word (tono normale, tono deciso, sottovoce/sussurrato, molto veloce) e 4 parole simili

scelte a piacere tra quelle dell'elenco dei negativi avversi. I partecipanti erano invitati a registrare in un ambiente silenzioso, senza TV, musica o rumori di sottofondo, per garantire la massima qualità dei campioni raccolti.

Caratteristiche delle registrazioni

- Wake word pronunciata: "Pixie" in italiano
- Diversità dei parlanti: età e genere
- Condizioni di registrazione: ambiente silenzioso, senza rumori di sottofondo e con il microfono a distanza ravvicinata. Le condizioni di rumore di fondo e le diverse distanze dal microfono non vengono richieste ai parlanti, ma simulate in seguito in modo controllato tramite data augmentation (rumore additivo e riverbero RIR).
- Formato audio finale: WAV, 16 kHz, mono, 16-bit PCM. I file originali ricevuti in altri formati, in particolare OGG/Opus dai messaggi vocali di WhatsApp, vengono convertiti in questo formato standard prima dell'inserimento nel dataset, tramite lo script `conversione.py`.

2.2 Rumori di Fondo

Per aumentare la robustezza del modello in ambienti reali, il dataset include campioni con diversi tipi di rumore di fondo sovrapposto alla wake word. I rumori di fondo provengono dai seguenti dataset pubblici, tutti ricampionati a 16 kHz: AudioSet (audioset_16k), FMA (fma), ESC-50 (ESC-50_16k), MUSAN (musan_16k) e UrbanSound8K (urbansound_16k). La tabella seguente riassume le categorie di rumore coperte:

Categoria	Esempi	Dataset
Ambiente domestico	TV in sottofondo, lavatrice, conversazioni lontane, cucina	AudioSet, ESC-50, MUSAN
Ambiente esterno	Traffico stradale, vento, voci in piazza	UrbanSound8K, ESC-50
Musica	Musica strumentale e con voce a volume variabile	FMA, MUSAN
Parlato sovrapposto	Altre voci sovrapposte non contenenti la wake word	MUSAN, AudioSet

Il mixing del rumore di fondo sui campioni audio non viene eseguito dal nostro codice: è la pipeline di augmentation di `openWakeWord` (comando `train.py --augment_clips`) a gestirlo in automatico, applicando il rumore a diversi livelli SNR per ciascun campione sintetico. I percorsi delle cartelle di rumore vengono semplicemente dichiarati nel parametro `background_paths` del file di configurazione YAML.

2.3 Room Impulse Response (RIR) per l'Augmentation

Per simulare l'effetto acustico di diverse stanze e ambienti chiusi, sono state utilizzate Room Impulse Response (RIR). La convoluzione di un segnale audio pulito con una RIR riproduce l'eco e il riverbero caratteristici di uno spazio fisico. Le RIR utilizzate nel progetto provengono da due dataset pubblici: **OpenSLR26** (openslr26_rirs) e **MIT RIR Dataset** (mit_rirs). Questi due dataset coprono una vasta gamma di ambienti acustici, dalle stanze di

piccole dimensioni fino a spazi aperti e ambienti grandi, garantendo una buona variabilità nelle condizioni di riverbero del dataset di addestramento.

In dettaglio, le tipologie di ambiente coperte includono:

- Stanze di piccole dimensioni (bagno, studio) — MIT RIR Dataset
- Stanze di medie dimensioni (salotto, ufficio) — MIT RIR Dataset e OpenSLR26
- Ambienti grandi (sala conferenze, corridoio) — OpenSLR26

La convoluzione dei campioni audio con le RIR non viene eseguita manualmente: è la stessa pipeline di augmentation di openWakeWord (comando `train.py --augment_clips`) a gestirla in automatico, ricevendo i percorsi dei dataset tramite il parametro `rir_paths` del file di configurazione YAML. Sia la convoluzione RIR che il mixing del rumore di fondo vengono applicati insieme in questa fase, producendo le varianti aumentate dei campioni sintetici (positivi e negativi avversi).

2.4 Registrazioni Generate con TTS

Una parte del dataset è costituita da campioni sintetici: lo scopo di questa fase non è semplicemente “ingrandire” il dataset, ma generare in modo controllato i samples sintetici, sia positivi (pronunce di “Pixie”) sia negativi avversi. Per la sintesi è stato usato PiperTTS, il motore TTS integrato nella libreria openWakeWord. PiperTTS è un sintetizzatore vocale open source ottimizzato per la generazione rapida di campioni audio localmente, senza la necessità di servizi cloud. La generazione è stata eseguita direttamente tramite il pipeline di addestramento di openWakeWord, che automatizza la produzione di campioni sintetici in varie voci e con diverse variazioni prosodiche. Va inoltre osservato che, nella generazione dei negativi avversi, openWakeWord non si limita alle parole indicate: tramite un phonemizer ne deriva automaticamente varianti foneticamente affini, aumentando così il numero complessivo dei negativi avversi sintetici a partire dalle parole proposte.

PiperTTS: caratteristiche e utilizzo

- Motore TTS: PiperTTS, integrato direttamente nel pipeline di openWakeWord per la generazione automatica dei campioni
- Variazione della velocità di pronuncia (rate) e del tono (pitch)
- Generazione di varianti con pausa prima/dopo la wake word

I campioni TTS sono stati sottoposti agli stessi processi di augmentation (rumore di fondo + RIR) applicati alle registrazioni reali.

2.5 Verifica delle Registrazioni

Prima di inserire i campioni nel dataset di addestramento, ciascuna registrazione è stata sottoposta a una fase di verifica che include:

Verifica delle registrazioni raccolte

- Ascolto manuale a campione per individuare registrazioni tagliate, silenziose o con artefatti
- Conversione in formato WAV PCM 16-bit / 16 kHz / mono tramite lo script `conversione.py` (necessaria soprattutto per i vocali WhatsApp ricevuti in OGG/Opus)

- Validazione del contenuto con `verifica_positivi_e_negativi_avversi.py`

Verifica delle registrazioni generate

- Validazione del contenuto con `verifica_positivi_e_negativi_avversi.py`
-

3. Raccolta del Dataset

3.1 Linguaggi e Ambienti di Sviluppo

Tecnologia	Utilizzo
Python 3.10,3.12	Linguaggio principale per scripting, preprocessing e training
Jupyter Notebook	Esplorazione del dataset e prototipazione rapida
Bash / Shell	Automazione pipeline e gestione file audio

3.2 Librerie Principali

Libreria	Funzione
librosa	Analisi e manipolazione di segnali audio
soundfile / sounddevice	Lettura, scrittura e registrazione audio
pydub	Conversione formati, normalizzazione e manipolazione audio
numpy / scipy	Elaborazione numerica, convoluzione RIR, DSP
PiperTTS	Generazione sintetica di campioni audio in italiano
openWakeWord	Libreria base per il riconoscimento della wake word
tfllite / onnx	Runtime per l'inferenza del modello

3.3 Programmi Sviluppati

Di seguito i principali script e moduli sviluppati per la gestione del dataset, con i riferimenti al repository GitHub del progetto.

conversione.py

Script di normalizzazione del formato audio. Converte in blocco tutti i file presenti nella cartella di input (formati supportati: WAV, MP3, FLAC, OGG, M4A) in WAV PCM 16-bit, mono, a 16 kHz, salvandoli in una cartella di output dedicata. È il primo passo della pipeline di preparazione del dataset: porta nello stesso formato standard sia i vocali raccolti dai partecipanti (spesso OGG/Opus da WhatsApp) sia eventuali campioni in altri formati, garantendo coerenza per le fasi successive.

Funzionalità principali:

- Caricamento e ricampionamento a 16 kHz mono tramite `librosa`

- Scrittura in PCM 16-bit con soundfile (subtype PCM_16), equivalente alla quantizzazione standard a interi a 16 bit attesa da openWakeWord
- Elaborazione batch dell'intera cartella con gestione degli errori per singolo file

verifica_lunghezza_samples.py

Script di controllo della durata dei singoli samples audio. Poiché il classificatore opera su finestre di durata fissa, ogni sample deve durare circa 1 secondo. Lo script legge istantaneamente i metadati di ciascun WAV (senza decodificare l'intero file) e, applicando una tolleranza tra 0,99 e 1,01 secondi, sposta automaticamente in una cartella di scarto i file con durata anomala, lasciando nella cartella originale solo i campioni validi. Al termine stampa un report con il numero di file corretti e scartati.

Funzionalità principali:

- Lettura veloce della durata dai soli metadati WAV (soundfile.info)
- Tolleranza di $\pm 0,01$ s per compensare gli arrotondamenti del campionamento
- Spostamento automatico dei file fuori standard in una cartella di scarto e report finale conteggiato

verifica_positivi_e_negativi_avversi.py

Script unico per la validazione automatica sia dei campioni positivi (pronunce di "Pixie") che dei negativi avversi (parole simili). Tramite un menù interattivo seleziona la modalità di verifica: Modalità 1 — Positivi (verifica che WhisperX trascriva la keyword come "pixie", "pixi", "piksi" o "picsi"); Modalità 2 — Negativi/Avversi (verifica che WhisperX trascriva una delle 10 parole avverse attese). Il modello ASR utilizzato è WhisperX, con supporto GPU/CPU automatico. I file non riconosciuti vengono spostati nella cartella audio_scartati e un report dettagliato viene salvato in report_falliti.txt. L'audio viene caricato una volta e salvato in un file di cache (.pkl) per evitare ricaricamenti nelle esecuzioni successive.

Funzionalità principali:

- Selezione interattiva della modalità: Positivi (keyword "Pixie") o Negativi/Avversi (10 parole foneticamente simili)
- Trascrizione con WhisperX (modello "base" di default, GPU se disponibile, altrimenti CPU con int8)
- Batching reale su GPU/CPU con batch_size configurabile (default 32, da ridurre a 4-8 con poca VRAM)
- Cache audio su disco (.pkl): il caricamento dei file WAV viene saltato nelle esecuzioni successive se la cache esiste già
- File non riconosciuti spostati automaticamente in audio_scartati/, con log dettagliato in report_falliti.txt (trascrizione effettiva di WhisperX per ogni fallimento)

Nota: il programma è attualmente in uso attivo per la validazione del dataset in corso.

Pipeline di generazione TTS (openWakeWord)

openWakeWord include un pipeline integrato per la generazione automatica di campioni sintetici di training. Dato il testo della wake word ("Pixie"), il pipeline utilizza PiperTTS per sintetizzare centinaia di pronunce in diverse voci, velocità di eloquio e toni. Per ciascuna pronuncia vengono generate ulteriori varianti tramite data augmentation: aggiunta di rumore di fondo a diversi livelli SNR (usando i dataset AudioSet, FMA, ESC-50, MUSAN, UrbanSound8K) e convoluzione con Room Impulse Response (OpenSLR26, MIT RIR). Il

risultato è un dataset sintetico di campioni positivi WAV a 16 kHz pronti per il training del classificatore. Vengono generate anche varianti con pausa prima e dopo la wake word, per insegnare al modello a rilevare la keyword indipendentemente dal contesto temporale.

□ **Repository GitHub:** <https://github.com/francescodemito04-crypto/SysAg-OWW>

4. Il Modello: openWakeWord

Per il riconoscimento della wake word "Pixie" è stato utilizzato openWakeWord, una libreria open source sviluppata da dscripka che fornisce un'architettura ottimizzata per il rilevamento di parole di attivazione su dispositivi con risorse limitate.

4.1 Architettura della Libreria

openWakeWord adotta un approccio a due stadi:

- Feature extraction: il segnale audio viene trasformato in mel-spectrogrammi tramite un modello audio embedding preaddestrato (Google Speech Embedding)
- Wake word classification: un classificatore leggero che opera sugli embedding per rilevare la presenza della wake word nel flusso audio

Questa separazione permette di addestrare solo il classificatore finale con il proprio dataset, mantenendo fisso il modello di feature extraction preaddestrato su grandi corpus audio.

4.2 Adattamento per "Pixie"

Il modello è stato adattato al riconoscimento specifico della parola "Pixie" in italiano attraverso le seguenti fasi:

- Preparazione del dataset nel formato richiesto da openWakeWord (file WAV 16 kHz mono)
- Fine-tuning del classificatore usando i campioni positivi (pronunce di "Pixie") e negativi (parlato generico, rumore)
- Configurazione delle soglie di attivazione per bilanciare sensibilità e specificità
- Esportazione del modello in formato TFLite per l'inferenza real-time

4.4 La Pipeline di Addestramento Automatico (Notebook)

L'intero addestramento è orchestrato dal notebook `automatic_model_training.ipynb`, un adattamento del notebook ufficiale di training automatico di openWakeWord personalizzato per la wake word "Pixie". Il notebook automatizza l'intero processo, dal reperimento dei dati fino all'esportazione del modello finale. La generazione dei campioni sintetici si appoggia a Piper (tramite la libreria `piper-sample-generator`), pertanto l'esecuzione completa è supportata solo su sistemi Linux. Le fasi principali, eseguite in sequenza, sono le seguenti:

1. Setup dell'ambiente: installazione di `piper-sample-generator`, di openWakeWord in modalità completa e delle dipendenze; download dei modelli pre-addestrati di embedding e mel-spectrogram

2. Download e conversione a 16 kHz dei dataset di rumore (AudioSet, FMA, ESC-50, MUSAN, UrbanSound8K) e delle Room Impulse Response (OpenSLR26, MIT RIR)
3. Download delle feature pre-calcolate: set di training ACAV100M (~2.000 ore) e set di validazione standard per i falsi positivi (~11 ore)
4. Conversione e pre-calcolo delle feature di Common Voice 26.0 italiano, rispettando gli split ufficiali del corpus (train per i negativi, dev+test per la validazione)
5. Costruzione di un validation_set ibrido unendo il set standard alla voce italiana di Common Voice
6. Definizione della configurazione di training tramite file YAML
7. Generazione dei campioni sintetici (positivi e negativi avversi) con Piper TTS
8. Data augmentation dei campioni (rumore di fondo additivo + convoluzione con RIR)
9. Addestramento del classificatore con early-stopping sulla metrica dei falsi positivi
10. Esportazione automatica del modello in formato ONNX e TFLite (con cella di conversione manuale come fallback)

4.5 Dati di Training

Per i campioni negativi di training il progetto adotta una soluzione ibrida che combina due fonti complementari: le feature pre-calcolate del dataset ACAV100M, che forniscono un'ampia diversità acustica generale (musica, rumori, lingue diverse), e le feature di Common Voice italiano, che insegnano specificamente al modello a non attivarsi sul parlato italiano "difficile". Per evitare il data leakage, i samples di Common Voice usati in training (split train) sono tenuti rigorosamente separati da quelli usati in validazione (split dev+test). All'interno di ogni batch il numero di finestre per classe è bilanciato esplicitamente, dando ampio spazio ai negativi per spingere il modello verso un basso tasso di falsi positivi.

Fonte	Ruolo nel training
ACAV100M (~2.000 h)	Negativi generici: diversità acustica generale
Common Voice IT (~10 h)	Negativi: rifiuto del parlato italiano
Negativi avversi (TTS+reali)	Negativi hard: parole foneticamente simili
Positivi (TTS+reali)	Positivi: pronunce della wake word

4.6 Configurazione del Training

L'intero training è guidato da un file di configurazione YAML, ottenuto modificando il modello di esempio di openWakeWord. La tabella seguente riassume i parametri principali impostati per il modello "Pixie".

Parametro	Valore impostato
target_phrase	"picsi" (grafia fonetica per ottenere la pronuncia di "Pixie" dal TTS)
custom_negative_phrases	Pizza, Pixel, Pyrex, Taxi, Dixit, Dixie, Pasticcio, Bixby, Pipsi, Mix
n_samples / n_samples_val	50.000 / 5.000
steps	50.000
max_negative_weight	300 (penalità sui falsi positivi)

layer_size	32 (dimensione del classificatore)
target_accuracy / recall	0,75 / 0,65
target_false_positives_per_hour	0,2 falsi positivi / ora
background_paths / rir_paths	AudioSet, FMA, ESC-50, MUSAN, UrbanSound8K / OpenSLR26, MIT RIR

Nota: la voce TTS attualmente impiegata per generare i positivi è un modello Piper inglese (libritts) con grafia fonetica “picsi”. L’uso di una voce TTS italiana è in valutazione (vedere i suggerimenti a fine documento).

4.7 Adattamenti Tecnici alla Libreria

Per far funzionare correttamente il pipeline di openWakeWord nel nostro ambiente, il notebook applica alcune patch mirate alla libreria:

- **Ricalibrazione di val_set_hrs:** nel sorgente la durata del validation set è cablata a 11,3 ore; viene sostituita automaticamente con la durata reale del nostro validation set custom, così che la metrica dei falsi positivi/ora sia corretta

5.4 Inferenza Real-Time: Finestra Scorrevole e Soglia

Durante l’uso reale il modello non analizza il segnale una volta sola: opera in **streaming continuo** tramite una finestra scorrevole. Il flusso audio dal microfono viene suddiviso in chunk da 80 ms (1.280 campioni a 16 kHz). Per ogni chunk il pipeline calcola i mel-spectrogram features e poi l’embedding, accumulando gli ultimi 16 chunk ($\approx 1,28$ secondi di contesto). Il classificatore riceve questi 16 embedding e produce un punteggio tra 0 e 1: se il punteggio supera la **soglia di attivazione** (threshold), il sistema dichiara che la wake word è stata pronunciata.

La soglia è il parametro più importante da calibrare in fase di deployment. Abbassarla aumenta il TPR (il modello reagisce più facilmente) ma aumenta anche i falsi positivi; alzarla riduce i falsi positivi ma rischia di far perdere pronunce a basso volume o parzialmente coperte da rumore. Il valore di default di openWakeWord è 0,5, ma per un ambiente reale si raccomanda di costruire una piccola curva ROC empirica variando la soglia da 0,3 a 0,9 su un campione di audio reale e scegliere il punto di funzionamento che rispetta i requisiti del caso d’uso.

Latenza: poiché il modello viene invocato ogni 80 ms, il ritardo massimo tra la fine della pronuncia e l’attivazione è di 80 ms, percettivamente impercettibile per l’utente. Il classificatore è abbastanza leggero (layer_size=32) da girare in tempo reale anche su hardware embedded senza GPU.

7. Struttura del Repository

```
├── .gitignore
├── automatic_model_training.ipynb
├── guida wsl-OWW.txt
├── my_model.yaml
```

- └─ Script controllo samples/
 - └─ conversione_wav.py
 - └─ inferenza modello onnx.py
 - └─ readme.txt
 - └─ verifica_lunghezza_samples.py
- └─ verifica_positivi_e_negativi_avversi.py

8. Riferimenti e Licenze dei Dataset

Di seguito i riferimenti completi a tutti i dataset, strumenti e librerie impiegati nel progetto, con l'indicazione della licenza applicabile.

8.1 Risorse

	Riferimento	Licenza
openWakeWord	github.com/dscripka/openWakeWord	Apache 2.0
Piper TTS / piper-sample-generator	github.com/rhasspy/piper-sample-generator	MIT
WhisperX	github.com/m-bain/whisperX	BSD-4-Clause

8.2 Dataset Audio

Dataset	Utilizzo	Riferimento	Licenza
Mozilla Common Voice 26.0	Negativi generici (parlato italiano)	commonvoice.mozilla.org — ardaillion.fr/common-voice-it	CC0 1.0
ACAV100M (features pre-calcolate)	Negativi generici (diversità acustica)	huggingface.co/datasets/davidscripka/openwakeword_features	Apache 2.0
Google AudioSet	Rumori di fondo	research.google.com/audioset — Gemmeke et al., ICASSP 2017	CC-BY 4.0
FMA (Free Music Archive)	Rumori di fondo (musica)	freemusicarchive.org — Defferrard et al., ISMIR 2017	CC-BY-NC 4.0 (subset small)
ESC-50	Rumori di fondo	github.com/karolpiczak/ESC-50 — Piczak, ACM MM 2015	CC-BY 3.0

MUSAN	Rumori di fondo e parlato	openslr.org/17 — Snyder et al., OpenSLR 2015	CC-BY 4.0
UrbanSound8K	Rumori di fondo urbani	urbansounddataset.weebly.com — Salamon et al., ACM MM 2014	Ricerca non commerciale
OpenSLR 26 (Simulated RIR)	Room Impulse Response	openslr.org/26 — Ko et al., INTERSPEECH 2017	Apache 2.0
MIT Environmental Impulse Responses	Room Impulse Response	huggingface.co/datasets/davidscripka/MIT_environmental_impulse_responses	CC-BY 4.0

9. Considerazioni finali

Il progetto è ancora in lavorazione. Le principali attività pianificate per le versioni successive del sistema e del dataset sono le seguenti:

- Integrazione dei vari script di controllo dei samples in un unico programma
- Aumento del numero di samples reali raccolti
- Aumento dei samples generici in lingua italiana
- Raccolta di samples reali positivi registrati anche con rumori di sottofondo
- Generazione dei samples sintetici tramite modelli TTS con pronuncia italiana
- Aumento delle variazioni e dei livelli di SNR applicati in fase di augmentation